PYSPARK COMPLETE NOTES

PHASE 0

Spark Foundations

Understand what Spark is, how it runs your code, how Databricks fits around it, and how to read a query plan.

YOU WILL

- ✓ Explain what Apache Spark and PySpark are and why DataFrames are used
- ✓ Name the driver, cluster manager, executor, task and partition and say what each does
- ✓ Tell transformations from actions and explain lazy evaluation
- ✓ Describe jobs, stages, tasks, shuffles and the DAG
- ✓ Find your way around a Databricks notebook and its cluster
- ✓ Say what Databricks adds to Apache Spark, and which notebook calls are Databricks-only
- ✓ Read a query plan with explain() and count jobs, stages and tasks

START HERE IF

Nothing - this is where you start. A little Python helps.

Built from a Databricks PySpark tutorial notebook and its course outline.
Clickable contents, links and bookmarks work in any PDF viewer.

Phase 0 · Spark Foundations

Understand what Spark is, how it runs your code, how Databricks fits around it, and how to read a query plan.

YOU WILL BE ABLE TO

- ✓ Explain what Apache Spark and PySpark are and why DataFrames are used
- ✓ Name the driver, cluster manager, executor, task and partition and say what each does
- ✓ Tell transformations from actions and explain lazy evaluation
- ✓ Describe jobs, stages, tasks, shuffles and the DAG
- ✓ Find your way around a Databricks notebook and its cluster
- ✓ Say what Databricks adds to Apache Spark, and which notebook calls are Databricks-only
- ✓ Read a query plan with explain() and count jobs, stages and tasks

START HERE IF

Nothing - this is where you start. A little Python helps.

IN THIS PDF

Explanations, diagrams and code from the notebook · sections tagged **Beyond the notebook** · a decoder for every term · interview questions · a data-engineering lens · a self-test with answers · a glossary

How to read the boxes

IN PLAIN WORDS A jargon-free explanation of the idea.	DECODE THE CODE Meaning of every function, option and symbol in the code above (first time it appears).
TIP A habit that makes your code cleaner, faster or safer.	WATCH OUT A trap that gives a wrong or surprising result.
NOTEBOOK OBSERVATION Something specific noticed in the course notebook.	DATA ENGINEERING LENS How the skill is used in real pipelines (summary page at the end).
EXTRA Extra knowledge that is not in the notebook.	BEYOND THE NOTEBOOK A tag under a heading: new material added to the notebook's topics, ending with an Interview corner.

CONTENTS (click an entry to jump to it)

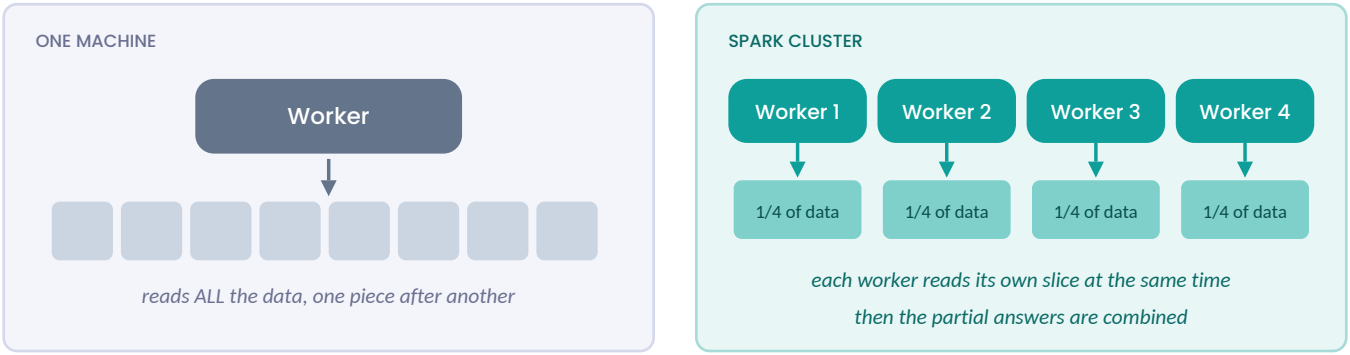
0.1 What is Apache Spark? . . . 3
0.2 Apache Spark architecture . . . 4
0.3 Lazy evaluation in Apache Spark . . . 5
0.4 Spark jobs, stages and tasks . . . 6
0.5 Spark DAG . . . 7
0.6 Databricks: free account, overview and notebook . . . 8
0.7 Spark cluster and data ingestion . . . 8
0.8 Apache Spark vs Databricks . . . 9
0.9 Notebooks and compute . . . 10
0.10 The Catalyst optimizer and explain() . . . 11
From notebook to pipeline . . . 15
Glossary . . . 16
Check yourself . . . 18
Answers and solutions . . . 19

0.1 What is Apache Spark?

IN PLAIN WORDS

Imagine counting the words in a library of a million books. One person reading alone takes years; a hundred people each reading a shelf and then adding up their counts finish quickly. Apache Spark is the software that splits a big data job across many machines (or CPU cores), lets them work in parallel, and combines the answers.

Apache Spark is an open-source, distributed data-processing engine. You describe *what* you want (filter these rows, total this column, join these tables) and Spark works out *how* to run it in parallel over a cluster. It keeps intermediate data in memory wherever it can, which makes multi-step jobs much faster than older disk-based systems such as Hadoop MapReduce.



Spark splits the data, works on the pieces in parallel, and combines the answers.

PySpark is the Python API for Spark. The same engine can also be driven from Scala, Java, R and SQL - the notebook uses PySpark and, at the very end, SQL.

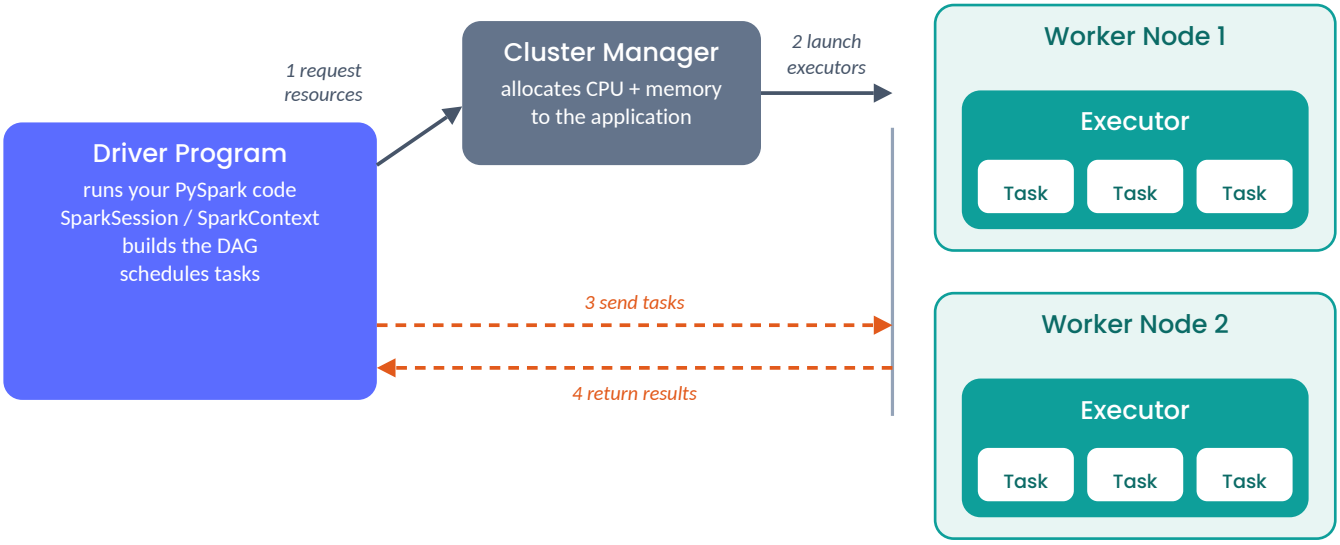
Spark component	What it is for
Spark Core	The engine: task scheduling, memory management, fault recovery. Everything else sits on top.
Spark SQL / DataFrames	Structured data with columns and types, SQL queries. **This is what the whole notebook uses.**
Structured Streaming	Processing data that keeps arriving (events, logs).
MLlib	Machine-learning algorithms that run in a distributed way.
GraphX	Graph processing (networks of nodes and edges).

WHY DATAFRAMES?

A DataFrame is a distributed table: named columns, a type for each column, and rows spread over the cluster. Because Spark knows the columns and types it can optimise your code automatically. Almost every notebook cell in this course is a DataFrame operation.

0.2 Apache Spark architecture

A Spark application is a team with a boss, a resource manager and workers:



Piece	Role	Everyday analogy
Driver	Runs your code, creates the SparkSession, turns your transformations into a plan (DAG) and schedules tasks.	The project manager
Cluster manager	Hands out machines, CPU and memory to the application.	The office administrator
Worker node	A machine in the cluster that hosts executors.	An office building
Executor	A process on a worker that runs tasks and holds data in memory for the application.	A team in the building
Task	The smallest unit of work: one operation on one partition of data.	One person's job slip
Partition	A slice of the data. Each partition is processed by one task.	One shelf of the library

What happens when you run a cell

1. Your PySpark code runs in the **driver**. Transformations only add steps to a plan.
2. When an **action** (like display) is called, the driver asks the **cluster manager** for resources.
3. **Executors** start on the worker nodes.
4. The driver splits the work into **tasks**, one per data partition, and sends them to executors.
5. Executors process their partitions in parallel and send results back to the driver.

> IN DATABRICKS YOU DO NOT MANAGE THIS
>
> Databricks creates and manages the cluster for you. A **SparkSession** called `spark` is already created and waiting in every notebook - that is why the notebook can start straight away with `spark.read` and `spark.sql`.

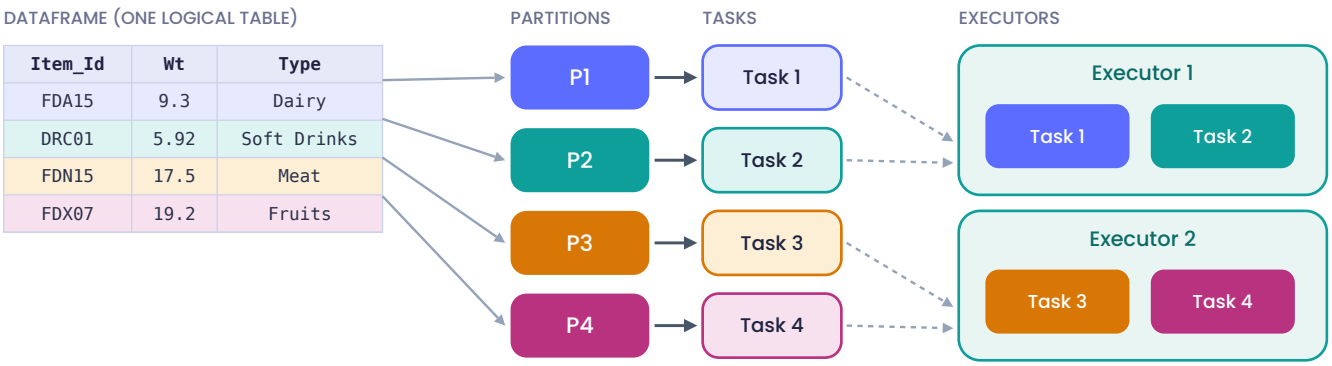
OUTSIDE DATABRICKS

On your own laptop you create the session yourself:

PYSPARK · ONLY NEEDED OUTSIDE DATABRICKS

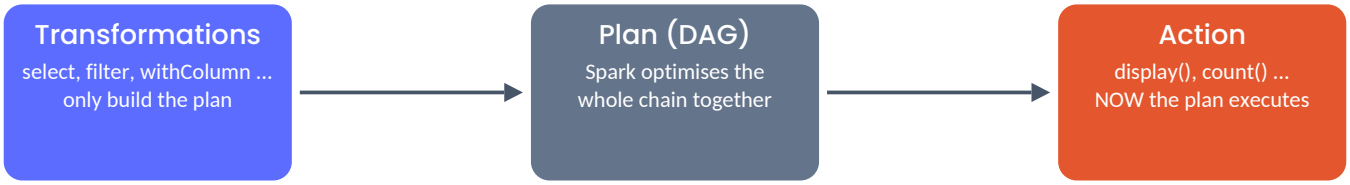
```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("my_app").getOrCreate()
```



A partition is a slice of the data. Spark runs one task per partition, and tasks run in parallel on the executors.

0.3 Lazy evaluation in Apache Spark



Nothing runs until an action is called - that is lazy evaluation.

Spark operations come in two kinds. **Transformations** describe a new DataFrame (`select`, `filter`, `withColumn`, `groupBy`, `join` ...). They return instantly because Spark only records the step. **Actions** ask for an actual result (`display()`, `count()`, `collect()`, writing a file ...). Only an action makes Spark run everything that was recorded.

PYSPARK · ILLUSTRATION USING NOTEBOOK COLUMNS

```
a = df.filter(col('Item_Type') == 'Soft Drinks')      # transformation - nothing runs
b = a.select('Item_Identifier', 'Item_MRP')           # transformation - nothing runs
b.display()                                           # ACTION - the whole chain runs now
```

DECODE THE CODE	
`.filter(condition)`	Keeps the rows where the condition is true - like SQL WHERE.
`col('name')`	Refers to a column by name so you can do something with it (rename, compare, calculate).
`==`	**Equals** test. A single **=** would assign instead of compare.
`.select(...)`	Keeps only the listed columns (or new calculated columns) - like SQL SELECT.
`.display()`	Databricks' table viewer. It is an **action**, so it makes Spark run the job.

Why is Spark lazy? Because it can look at the *whole* chain before running it and optimise: skip columns you never use, apply filters as early as possible, and combine steps. If it ran every line immediately it could not plan ahead.

Transformations (lazy)	Actions (run the job)
select, filter, withColumn, withColumnRenamed, drop, dropDuplicates, distinct, sort, limit, union, groupBy + agg, pivot, join	display(), show(), count(), collect(), and writing data with **.write ... .save()** / **saveAsTable()**

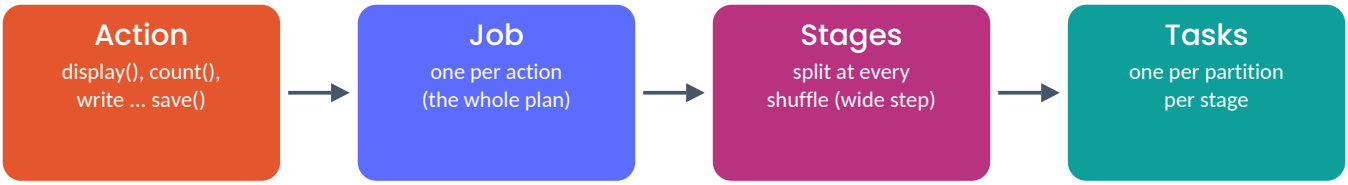
Transformations are also split by how much data movement they need:

Type	Meaning	Examples from the notebook
Narrow	Each output partition depends on one input partition. No data moves between machines.	select, filter, withColumn, drop, limit (per partition)
Wide	Output needs data from many partitions, so rows are **shuffled** across the network (expensive).	groupBy, join, sort, distinct / dropDuplicates, pivot, window functions

DATAFRAMES ARE IMMUTABLE

A transformation never changes a DataFrame - it returns a **new** one. That is why the notebook writes `df = df.withColumn(...)` when it wants to keep a change, and why a bare `df.drop('Item_Visibility').display()` leaves `df` untouched.

0.4 Spark jobs, stages and tasks



An action triggers a job; the job is cut into stages at shuffles; each stage runs as parallel tasks.

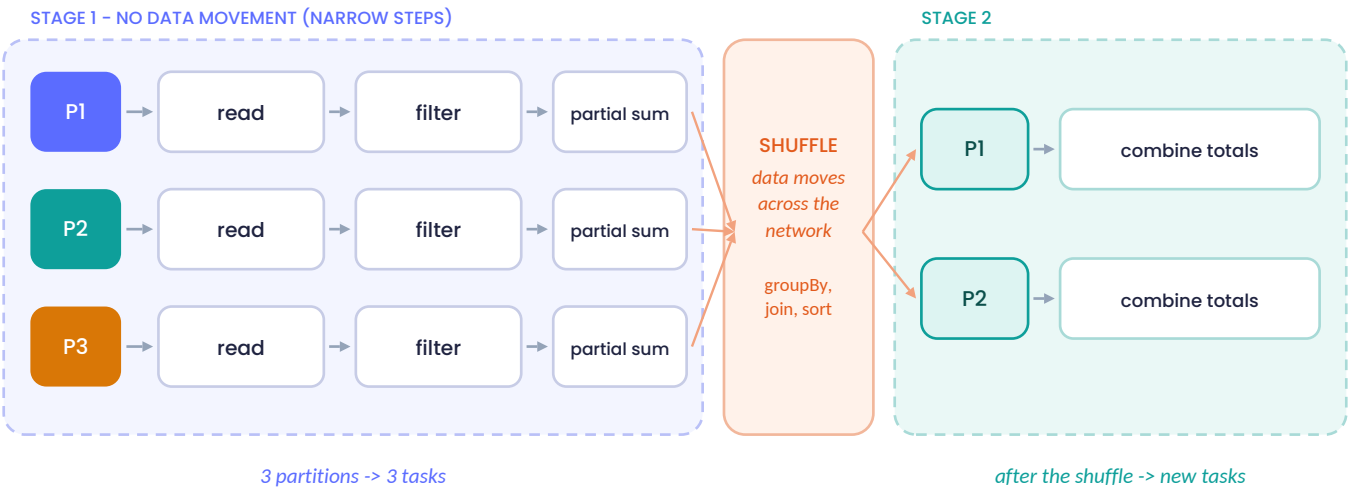
Term	Definition	Example
Job	Everything Spark does because of **one action**.	Calling `display()` on a chain of transformations.
Stage	A part of a job that can run without shuffling data. A new stage begins at every wide transformation.	`groupBy('Item_Type').agg(...)` = stage 1 (read + partial sums) and stage 2 (combine after the shuffle).
Task	One stage applied to one partition. Tasks of a stage run in parallel.	A stage over 8 partitions runs 8 tasks.

WHERE TO LOOK

In Databricks, click the job links under a cell's output (or open the Spark UI) to see the jobs, their stages and the number of tasks. Watching this view while you run the notebook is the fastest way to make these ideas real.

SHUFFLE PARTITIONS

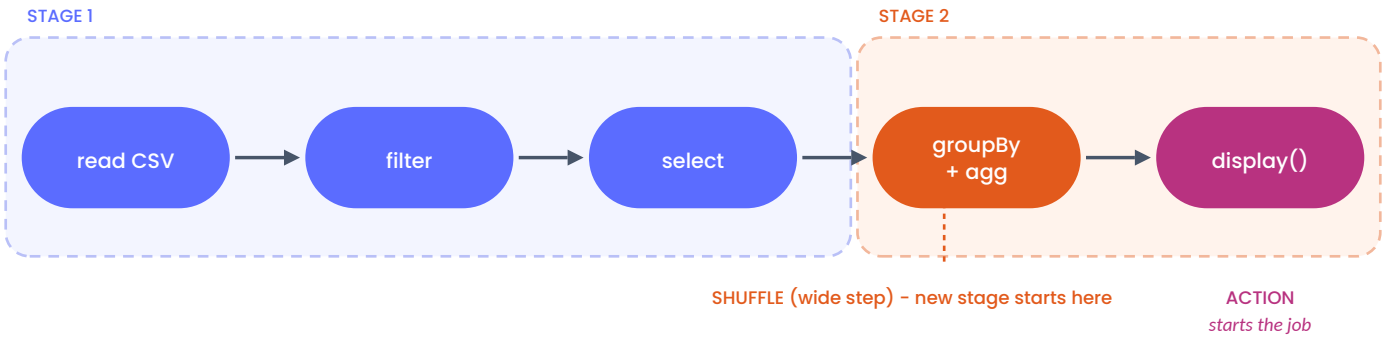
After a shuffle Spark uses `spark.sql.shuffle.partitions` partitions (default 200), so a post-shuffle stage can have up to 200 tasks. Adaptive Query Execution, which Databricks enables, may reduce this automatically.



A wide transformation (shuffle) cuts the job into a new stage. All tasks of one stage run in parallel.

0.5 Spark DAG

DAG stands for **Directed Acyclic Graph**: a set of steps (nodes) joined by arrows (edges) that go one way and never loop back. When you chain transformations, Spark builds this graph of steps - it is the plan it will execute when an action arrives. The DAG is what lets Spark split a job into stages (cut the graph at every shuffle) and re-run just the lost part if a machine fails.



The DAG: directed (one way), acyclic (no loops) graph of steps. Spark cuts it into stages at every shuffle.

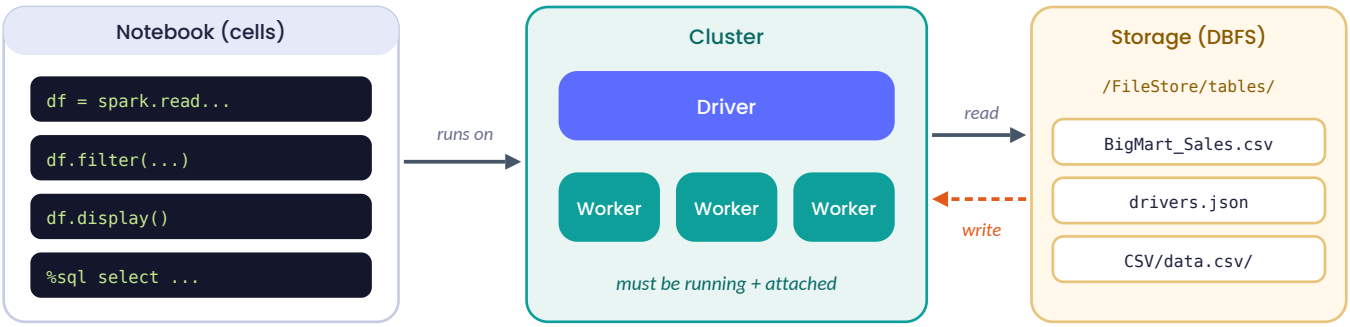
- **Directed** - work flows from the data source to the result.
- **Acyclic** - there are no loops, so the plan always finishes.
- **Graph** - nodes are operations (read, filter, groupBy ...), edges are the data flowing between them.

> **SEEING A DAG**
> Open a finished job in the Spark UI and click *DAG Visualization*. Each box is a stage and each dot inside it is an operation. Try it on the `groupBy` cell: you will see the stage boundary exactly where the shuffle happens.

> **READING A CSV MAY START A JOB BY ITSELF**
> With `inferSchema=True` Spark must scan the file to guess column types, so even a pure read can show a job. Supplying your own schema (Phase 1) avoids that extra pass.

0.6 Databricks: free account, overview and notebook

Databricks is the cloud platform where Spark runs in this course. A free account gives you a workspace with a small cluster, enough to practise everything in these notes.



A Databricks notebook runs on an attached cluster, and the cluster reads and writes files in storage.

Part of the workspace	What it does
Workspace	Where your notebooks and folders live.
Compute / cluster	The Spark cluster. It must be **running** and your notebook must be **attached** to it before cells can run.
Data / DBFS	File storage. The notebook reads and writes under `/FileStore/tables/`.
Notebook	An ordered set of cells. Each cell is code (Python by default) or Markdown text.

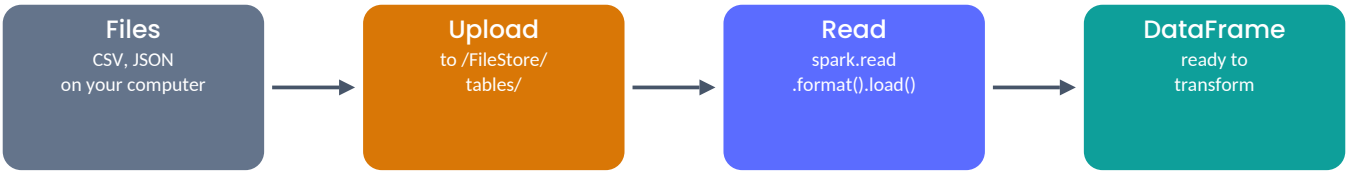
Notebook features used in the course

Feature	Meaning
`spark`	The ready-made SparkSession.
`display(df)` / `df.display()`	Databricks' rich table viewer (it is an action). Outside Databricks use `df.show()`.
`dbutils`	Databricks utilities, for example `dbutils.fs.ls(path)` to list files.
`%sql`	A magic command that turns a cell into SQL (used in Phase 7).
Markdown cells	Text starting with `#`, `###` or `####` for headings. The notebook is organised with them.

> **YOUR FILE PATHS MAY DIFFER**
>
> The notebook reads from `/FileStore/tables/...` (DBFS), which matches the classic Databricks free workspace. Newer free workspaces may store uploads in Unity Catalog Volumes with paths like `/Volumes/<catalog>/<schema>/<volume>/file.csv`. If a read fails with *path does not exist*, list the folder with `dbutils.fs.ls(...)` and copy the exact path shown.

0.7 Spark cluster and data ingestion

A **Spark cluster** is the group of machines (driver plus workers) that executes your notebook. **Data ingestion** simply means getting data into the platform so Spark can process it: uploading files, reading from databases, or streaming events in. In this course ingestion is the simplest form - you upload the CSV and JSON files to the workspace and Spark reads them from storage, which is exactly what Phase 1 does.

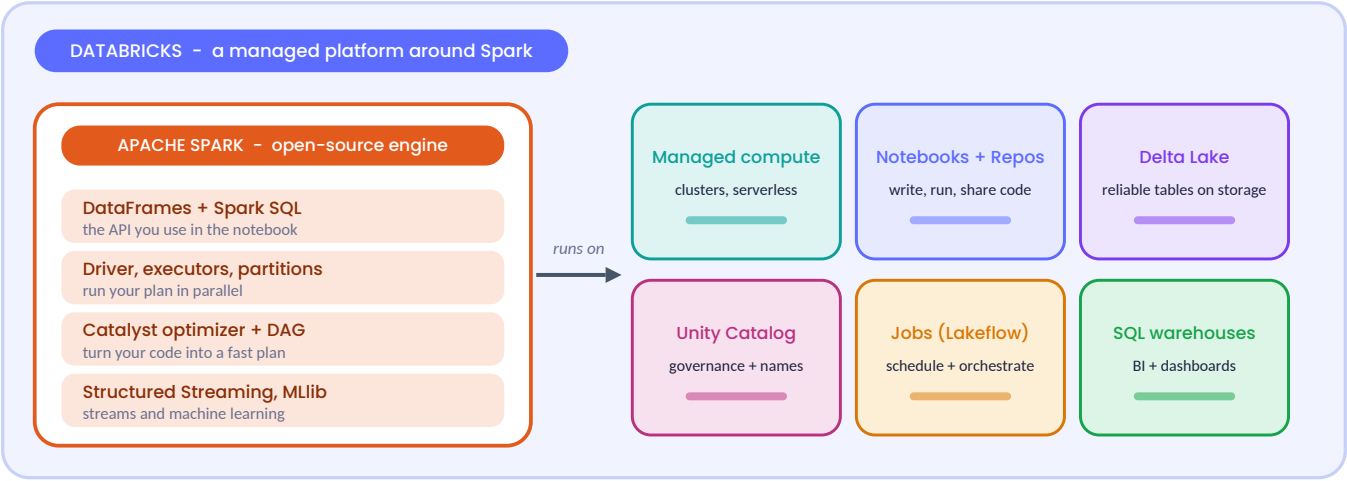


Data ingestion = getting data into the platform so Spark can read it. In this course: upload, then spark.read.

0.8 Apache Spark vs Databricks

BEYOND THE NOTEBOOK

The notebook runs on **Databricks**, but the code you write in it is **PySpark**. They are two different things, and knowing which is which tells you what will still work on your laptop, on another cloud, or only inside Databricks.



Spark is the engine. Databricks is the platform that runs, stores, governs and schedules it.

	Apache Spark	Databricks
What it is	An open-source engine that processes data in parallel.	A cloud platform built around Spark, created by Spark's original developers.
What you get	DataFrame, SQL, streaming and ML APIs plus the driver / executor engine.	Spark **plus** managed compute, notebooks, Delta Lake, Unity Catalog, Jobs and SQL warehouses.
Who runs the machines	You (or your platform team): install, size, patch, scale.	Databricks creates, scales and stops the compute for you.
Tables and storage	Files, or tables you configure yourself.	Delta tables by default, registered in Unity Catalog.
Governance	You build it.	Permissions, lineage and auditing in Unity Catalog.
Scheduling	Bring your own scheduler (Airflow, cron ...).	Built-in Jobs (now called Lakeflow Jobs).

Where the notebook uses Databricks-only conveniences

In the notebook	What it really is	Portable alternative
`df.display()`	Databricks' rich table viewer.	`df.show()` prints a text table on any Spark.
`%sql` cell	A Databricks notebook feature.	`spark.sql("...")` works everywhere.
`dbutils.fs.ls(...)`	Databricks file utilities.	Ordinary file tools, or the Hadoop file-system API.
`/FileStore/tables/...`	A Databricks (DBFS) path.	A local, `s3://`, `abfss://` or `gs://` path.
`spark` already exists	Databricks creates the session for you.	`SparkSession.builder.getOrCreate()`
`saveAsTable(...)` makes a Delta table	Databricks' default table format.	Open-source Spark defaults to Parquet unless Delta Lake is installed.

PYSPARK · PORTABLE VS DATABRICKS-ONLY

```
df.show(5)       # works on any Spark: prints the first 5 rows as text
df.display()     # Databricks only: interactive table, charts, download
```

DECODE THE CODE

`.show(n, truncate)`	Prints the first rows as text in any Spark environment; *truncate=False* stops long values being cut.

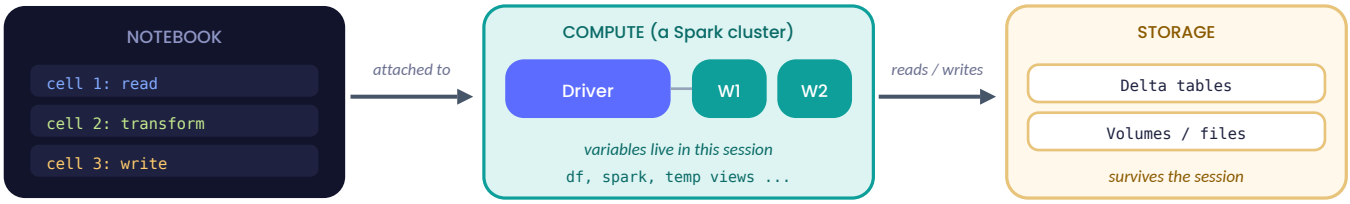
WHY THIS MATTERS FOR A DATA ENGINEER

Keep the **logic** (reads, transformations, joins) in plain PySpark and keep Databricks-only calls at the edges (display, dbutils, paths). That code is easier to test, to move, and to explain in an interview.

0.9 Notebooks and compute

BEYOND THE NOTEBOOK

A Databricks **notebook** is only a document of cells. It cannot run anything by itself: it must be **attached to compute**, a Spark cluster that provides the driver and executors from Phase 0. Variables such as `df` live in that attached session.



THREE KINDS OF COMPUTE YOU WILL MEET

All-purpose
Interactive. You start it, attach notebooks, share it. Good for learning and exploring; stays running until stopped.

Job compute
Created for one automated run, then terminated. Cheaper; the right choice for production pipelines.

Serverless
Databricks manages the machines. Starts in seconds, no cluster to size. Free Edition uses serverless only.

Detach or restart the compute and the session state is gone: that is why a temp view disappears but a saved table does not.

Compute type	Who starts it	Lives	Best for
All-purpose compute	You	Until you stop it (or it auto-terminates)	Interactive notebooks, exploring, learning.
Job compute	The job itself	One automated run, then it terminates	Scheduled pipelines; cheaper than all-purpose.
Serverless compute	Databricks, instantly	Managed for you	Notebooks and jobs without sizing a cluster. Free Edition uses serverless only.
SQL warehouse	You or Databricks	Managed	SQL queries and dashboards (it runs SQL, not PySpark notebooks).

SERVERLESS WORKS A LITTLE DIFFERENTLY

On serverless compute (all of Free Edition) some things in these notes behave differently, according to Databricks' documentation:

cache() / persist() are not supported, the **RDD API** (`df.rdd`) is not available, the **Spark UI** is replaced by the query profile, most `spark.conf.set` settings cannot be changed, and access to the old DBFS root is limited - use **Volumes** instead (Phase 6). Where a later section depends on this, it says so and gives an alternative.

PYSPARK · CHECK WHAT YOU ARE ATTACHED TO

```
spark.version                                  # Spark version of the attached compute
spark.range(3).display()            # a 3-row DataFrame: the 'hello world' of Spark
```

NOTEBOOK HYGIENE

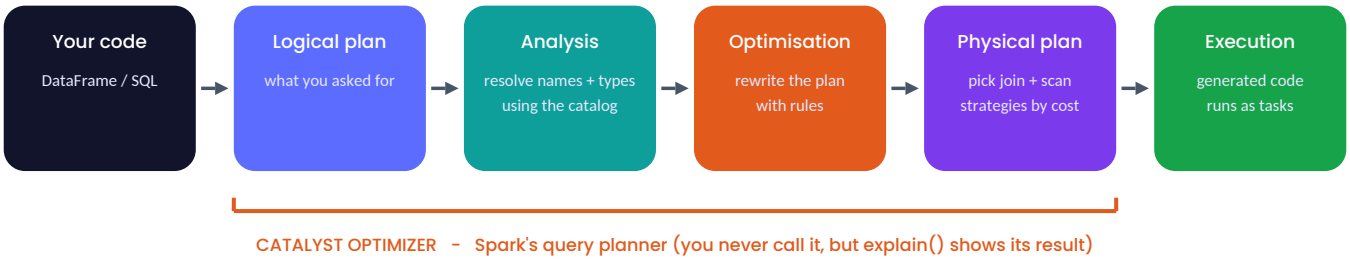
Run all from the top before trusting a result. A notebook remembers every variable, so an old value can hide a bug in a cell you edited.

A temporary view or DataFrame disappears when the session ends; a **saved table** does not. That is why Phase 6 matters for pipelines.

0.10 The Catalyst optimizer and explain()

BEYOND THE NOTEBOOK

You tell Spark *what* you want; **Catalyst**, Spark's query optimizer, works out *how*. It rewrites your plan before any data is read, so the order in which you wrote lazy steps is often not the order in which Spark runs them.



WHAT THE OPTIMISER DOES FOR YOU – EXAMPLES

Predicate pushdown
move filter() as early as possible, even into the file reader

Column pruning
read only the columns the query really uses

Constant folding
work out 1 + 2 once at planning time

Join strategy
choose broadcast or sort-merge by data size

AQE (Adaptive Query Execution) goes one step further: it re-plans WHILE the job runs, using the real sizes of shuffle results.

PYSPARK · PRINT THE PLAN WITHOUT RUNNING THE JOB

```
q = df.filter(col('Item_MRP') > 100)\
      .groupBy('Item_Type')\
      .agg(sum('Item_MRP').alias('Total_MRP'))

q.explain()
```

DECODE THE CODE

`\ (backslash)`	Line continuation: the statement carries on in the next line. Nothing may follow it on the same line.
`.groupBy(columns)`	Collects rows that share the same key(s) into groups. Needs an aggregate next.
`.agg(...)`	Calculates one or more aggregates for every group.
`sum(col)`	Total of a column (Spark's own *sum*, available through the import *).
`.alias('new')`	Gives a column a new name **in this result only**.
`.explain()`	Prints the plan Spark will run - **without** running it. *mode='formatted'* gives an easier outline.

OUTPUT · SIMPLIFIED (run on Spark 4.0.1; internal ids like #74 and long paths removed)

```
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- HashAggregate(keys=[Item_Type], functions=[sum(Item_MRP)])
   +- Exchange hashpartitioning(Item_Type, 200)
      +- HashAggregate(keys=[Item_Type], functions=[partial_sum(Item_MRP)])
         +- Filter (isnotnull(Item_MRP) AND (Item_MRP > 100.0))
            +- FileScan csv [Item_Type,Item_MRP] PushedFilters: [IsNotNull(Item_MRP),
               GreaterThan(Item_MRP,100.0)], ReadSchema: struct<Item_Type:string,Item_MRP:double>
```

Read the plan **from the bottom up**: Spark scans the file, filters, pre-aggregates inside each partition, shuffles (**Exchange**), then finishes the aggregation. Notice the scan reads only **two columns** and already carries the filter: that is **column pruning** and **predicate pushdown**, done for you.

Call	What it prints
`df.explain()`	The **physical plan** only - what will actually run.
`df.explain(True)` or `mode='extended'`	Four plans: parsed, analyzed, optimized logical, and physical. Shows what Catalyst changed.
`df.explain(mode='formatted')`	The physical plan as a numbered outline with details per step - the easiest to read.
`df.explain(mode='cost')`	The logical plan with size estimates, where statistics exist.

PYSPARK · THE OPTIMIZER REORDERS NARROW STEPS

```
a = df.select('Item_Type', 'Item_MRP').filter(col('Item_MRP') > 100)
b = df.filter(col('Item_MRP') > 100).select('Item_Type', 'Item_MRP')

a.explain()      # the optimizer produces the SAME plan for a and b
b.explain()
```

WHAT TO LOOK FOR IN A PLAN

Exchange = a shuffle, so a new stage. Count them: fewer is usually faster.

BroadcastHashJoin vs **SortMergeJoin** tells you how a join will run (Phase 4).

PushedFilters and **ReadSchema** show how much of the file is really read.

AdaptiveSparkPlan at the top means **AQE** is on: Spark may change the plan while the job runs, using real data sizes. This plan says `isFinalPlan=false` because nothing has run yet.

EXPLAIN() IS NOT AN ACTION

It only prints the plan. Nothing is read and no job starts. For the plan that *actually ran*, open the query in the Spark UI SQL tab (on serverless: the query profile).

PUSHDOWN DEPENDS ON THE FILE FORMAT

With Parquet and Delta, pushed filters let Spark **skip** data using file statistics. With CSV the filter is applied while parsing, but the whole file is still read, which is one more reason to move from CSV to Parquet / Delta (Phase 6).

Worked example: count the jobs, stages and tasks

PYSPARK · ONE PIPELINE, ONE ACTION

```
df = spark.read.format('csv').option('header', True).load('/FileStore/tables/BigMart_Sales.csv')

out = df.filter(col('Item_MRP') > 100)\
        .groupBy('Item_Type')\
        .agg(sum('Item_MRP'))

out.display()
```

DECODE THE CODE

`spark.read`	Starts a **DataFrameReader**: the object you configure and then use to load a file.
`.format('csv')`	Chooses the file format: **csv**, **json**, **parquet** ... (the same call is used when writing).
`.option(key, value)`	Sets **one** setting for the reader or writer. Call it again for each extra setting.
`header`	**True** = the first row of the CSV holds column names, not data.
`.load(path)`	Points Spark at the file or folder and returns a DataFrame. (Lazy until an action needs the data.)
`/FileStore/tables/`	The Databricks folder (DBFS) where uploaded files live.

Question	Answer	Why
How many **jobs**?	1	Only `display()` is an action. The read, filter and groupBy just build the plan.
How many **stages**?	2	The groupBy needs a shuffle: stage 1 = scan, filter, partial aggregate; stage 2 = final aggregate.
How many **tasks**?	Stage 1: one per input partition. Stage 2: one per shuffle partition.	A small CSV is often a single input partition. The shuffle uses `spark.sql.shuffle.partitions` (200 by default), though AQE may merge small ones.
Which steps are **narrow** / **wide**?	filter = narrow; groupBy = wide	Narrow steps work inside one partition; wide steps move rows between partitions.

INTERVIEW CORNER

Q Explain Spark's architecture in one minute.

A A **driver** runs your program, builds a plan (DAG) and asks a **cluster manager** for resources. **Executors** on worker nodes run **tasks**, one per data **partition**, and keep data in memory. Results go back to the driver.

Q What is lazy evaluation and why does Spark use it?

A Transformations only record a plan; an **action** triggers execution. Because Spark sees the whole plan first, Catalyst can reorder, prune columns and push filters down before reading any data.

INTERVIEW CORNER

Q Transformation versus action? Narrow versus wide?

A Transformations (select, filter, groupBy, join) return a new DataFrame lazily; actions (display, count, write) run the job. Narrow transformations work inside one partition; **wide** ones (groupBy, join, sort, distinct) need a **shuffle**.

Q What is the difference between a job, a stage and a task?

A One **action** = one **job**. A job is split into **stages** at each shuffle. A stage runs one **task** per partition, in parallel.

Q Why is a shuffle expensive?

A Rows are written to disk, sent across the network, and read back by other executors; the stage must also wait for its slowest task. It is the main cost in most Spark jobs.

Q What does the Catalyst optimizer do, and how do you inspect it?

A It turns your code into a logical plan, optimizes it with rules (predicate pushdown, column pruning, constant folding, join strategy) and picks a physical plan. Use `df.explain()` (or `'formatted'`) to see the result.

Q Apache Spark or Databricks - what is the difference?

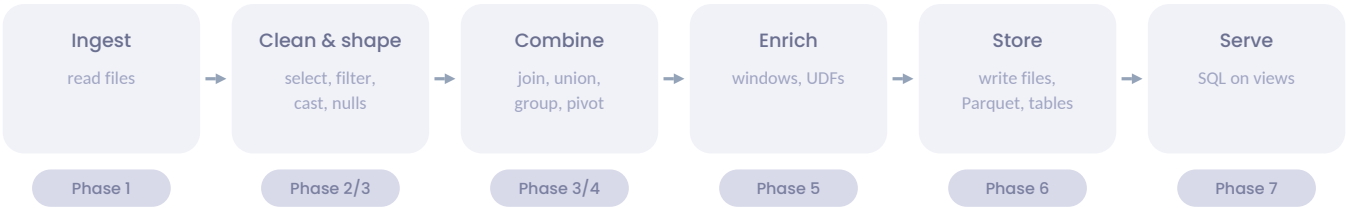
A Spark is the open-source engine. Databricks is a managed platform around it: compute, notebooks, Delta Lake, Unity Catalog and Jobs. PySpark code is portable; `display`, `dbutils` and DBFS paths are Databricks-specific.

PHASE 0 CHECKPOINT

- I can explain what Spark does and why DataFrames are used.
- I can name the driver, cluster manager, executor, task and partition and say what each does.
- I know transformations are lazy and only an action runs the plan.
- I can tell a job from a stage from a task, and I know a shuffle creates a new stage.
- I know what a DAG is and where to look at it in the Spark UI.
- My notebook is attached to a running cluster and I know my file paths.
- I can say what Spark does and what Databricks adds, and name the Databricks-only calls in the notebook.
- I know a notebook must be attached to compute, and I can tell all-purpose, job and serverless compute apart.
- I can print a plan with `explain()`, read it bottom-up and spot Exchange, PushedFilters and the join type.
- I can count jobs, stages and tasks for a small pipeline.

From notebook to pipeline

Data engineers use these same skills inside **pipelines**. Here is where this phase sits, and how it is used in practice.



DATA ENGINEERING LENS

A data pipeline is **ingest, transform, write**. The notebook is organised in exactly that order: read data (Phase 1), transform it (Phases 2-5), write it (Phase 6) and query it (Phase 7).

Because Spark is lazy, a pipeline does nothing until an **action** runs. In a pipeline, the write at the end is the action that makes the whole chain execute.

Shuffles are where the cost is. groupBy, join, sort and distinct create new stages; when a job is slow, look at its stage boundaries in the Spark UI.

Partitions decide how much work can run in parallel: one task per partition.

Beyond the notebook: keep the **logic in plain PySpark** and Databricks-only calls (display, dbutils, DBFS paths) at the edges - the code is then portable and testable.

Read the plan with **explain()** before and after a change; count the Exchange lines.

Glossary

Concepts from this phase, then every **code term** the decoder explained, in the order you met them.

Concept	In plain words
Apache Spark	Open-source engine that splits big data jobs across many machines and combines the results.
PySpark	The Python API for Spark.
DataFrame	A distributed table: named columns, a type per column, rows spread over the cluster.
SparkSession	Your entry point to Spark; in Databricks it already exists as **spark**.
Driver	Runs your code, builds the plan (DAG) and schedules tasks.
Cluster manager	Hands out machines, CPU and memory to the application.
Worker node / Executor	A machine in the cluster / the process on it that runs tasks and holds data.
Partition	A slice of the data. One task processes one partition.
Task / Stage / Job	Task = one operation on one partition. Stage = tasks that run without a shuffle. Job = everything triggered by one action.
Transformation	A lazy step that describes a new DataFrame (select, filter ...).
Action	A step that asks for a result and runs the plan (display, write ... save).
Lazy evaluation	Spark only records transformations and runs them when an action is called.
Narrow / Wide transformation	Narrow: no data movement between partitions. Wide: needs a shuffle (groupBy, join, sort).
Shuffle	Moving rows across the network so rows with the same key meet; it creates a stage boundary.
DAG	Directed Acyclic Graph: the one-way, loop-free plan of steps Spark builds from your code.
Databricks / DBFS	The cloud platform used in the course / its file storage (e.g. /FileStore/tables/).
Data ingestion	Getting data into the platform so Spark can read it.
Catalyst	Spark's query optimizer: rewrites your plan (pushdown, pruning, join choice) before running.
explain()	Prints the plan Spark will run, without running it.
AQE	Adaptive Query Execution: Spark adjusts the plan while the job runs, using real data sizes.
Compute / cluster	The machines a notebook is attached to; all-purpose, job or serverless.
Databricks	A managed cloud platform around Spark with notebooks, Delta, Unity Catalog and Jobs.

Code terms decoded in this phase

Code term	Meaning
`.filter(condition)`	Keeps the rows where the condition is true - like SQL WHERE.
`col('name')`	Refers to a column by name so you can do something with it (rename, compare, calculate).
`==`	**Equals** test. A single **=** would assign instead of compare.
`.select(...)`	Keeps only the listed columns (or new calculated columns) - like SQL SELECT.
`.display()`	Databricks' table viewer. It is an **action**, so it makes Spark run the job.
`.show(n, truncate)`	Prints the first rows as text in any Spark environment; *truncate=False* stops long values being cut.
`\ (backslash)`	Line continuation: the statement carries on in the next line. Nothing may follow it on the same line.
`.groupBy(columns)`	Collects rows that share the same key(s) into groups. Needs an aggregate next.
`.agg(...)`	Calculates one or more aggregates for every group.
`sum(col)`	Total of a column (Spark's own *sum*, available through the import *).
`.alias('new')`	Gives a column a new name **in this result only**.
`.explain()`	Prints the plan Spark will run - **without** running it. *mode='formatted'* gives an easier outline.
`spark.read`	Starts a **DataFrameReader**: the object you configure and then use to load a file.
`.format('csv')`	Chooses the file format: **csv**, **json**, **parquet** ... (the same call is used when writing).
`.option(key, value)`	Sets **one** setting for the reader or writer. Call it again for each extra setting.
`header`	**True** = the first row of the CSV holds column names, not data.
`.load(path)`	Points Spark at the file or folder and returns a DataFrame. (Lazy until an action needs the data.)
`/FileStore/tables/`	The Databricks folder (DBFS) where uploaded files live.

Check yourself

Try each question first, then click **Show answer**. Every answer has a **back** link that returns you to the question.

Questions

Q1 What does the **driver** do, and what does an **executor** do? Show answer

Q2 Why does `df.filter(col('Item_Type') == 'Meat')` not run anything by itself? Show answer

Q3 Name two actions and three transformations used in the notebook. Show answer

Q4 What makes Spark start a **new stage**? Show answer

Q5 How are partitions and tasks related? Show answer

Q6 Apache Spark or Databricks: which provides `display()`, and which provides the DataFrame engine? Show answer

Q7 In `df.filter(...).groupBy(...).agg(...).display()` how many jobs and stages are there, and why? Show answer

Q8 What do an **Exchange** line and **PushedFilters** in a plan tell you? Show answer

Hands-on tasks

Task 1 Run a cell that lists the files in the uploads folder. Which two data files does the notebook use? Show solution

Task 2 Predict, then check: after running `df.drop('Item_Visibility').display()`, does `df` still contain Item_Visibility? How do you keep the change? Show solution

Task 3 Print the plan for a filter + groupBy on the BigMart DataFrame and count the Exchange lines. Show solution

Answers and solutions

A1 The driver runs your code, builds the plan (DAG) and schedules the tasks. Executors run the tasks on the worker nodes and hold data in memory. back to question

A2 filter is a **transformation**, which is lazy: Spark only records it. An action such as `display()` makes Spark run the recorded plan. back to question

A3 Actions: `display()`, and writing with `.save()` / `.saveAsTable()`. Transformations: for example `select`, `filter`, `withColumn`. back to question

A4 A **shuffle**, caused by a wide transformation such as groupBy, join, sort or distinct. Narrow steps (select, filter, withColumn) stay in the same stage. back to question

A5 Each stage runs **one task per partition**, and the tasks of a stage run in parallel on the executors. back to question

A6 display() is Databricks-only (use show() elsewhere); the DataFrame engine is Apache Spark, which Databricks packages with managed compute, Delta, Unity Catalog and Jobs. back to question

A7 **One job** (one action) and **two stages**: the groupBy needs a shuffle, which ends stage 1. back to question

A8 Exchange = a shuffle (a stage boundary). PushedFilters = filters applied while reading, so less data is scanned. back to question

Task solutions

Solution 1 back to task

SOLUTION

```
dbutils.fs.ls('/FileStore/tables/')
# the notebook reads BigMart_Sales.csv and drivers.json
```

Solution 2 back to task

SOLUTION

```
df.drop('Item_Visibility').display()   # df is unchanged (transformations return a new DataFrame)

df = df.drop('Item_Visibility')         # assign it back to keep the change
```

Solution 3 back to task

SOLUTION

```
q = df.filter(col('Item_MRP') > 100).groupBy('Item_Type').agg(sum('Item_MRP'))
q.explain()      # one Exchange = one shuffle = two stages
```

Start of the series

NEXT
Phase 1 · Reading Data →

Tip: keep all nine PDFs in the same folder so the previous / next links open the right file.